Overview

Deploying LLM applications is not just shipping an API endpoint. Production success requires orchestration across serving infrastructure, observability signals, evaluation workflows, and governance controls.

This chapter covers an operational blueprint for LLM applications in production.

Textual architecture diagram:

```
Client -> API Gateway -> Orchestrator -> (Retriever + Tools + LLM) -> Post-Processor ->
Policy Guardrails -> Response
```

With side channels:

```
Tracing + Metrics + Evaluation + Governance Logs
```

Deployment Architecture for LLM Applications

Layered architecture

1. **Interface layer**: HTTP/gRPC endpoints, auth, rate limits.
2. **Orchestration layer**: routing, prompt assembly, tool invocation.
3. **Model layer**: hosted/open model runtime.
4. **Context layer**: vector retrieval and knowledge sources.
5. **Policy layer**: content filtering, safety and compliance checks.
6. **Observability layer**: telemetry and evaluation pipeline.

Release strategies

- Shadow runs for quality comparison.
- Canary rollout by user segment or traffic percentage.
- Blue-green for fast operational rollback.

Incident-aware design

Mandatory production capabilities:

- timeout and retry policy,
- fallback response path,
- circuit breaker for upstream model/provider outages,
- explicit degraded-mode behavior.

Observability for LLM Applications

What to trace

For each request, capture:

- prompt version,
- model identifier and parameters,
- retrieval metadata (top-k docs, scores),
- tool-call sequence,
- token usage and latency,
- safety filter outcomes.

Metrics layers

Infrastructure metrics

- p95/p99 latency,
- error and timeout rates,
- throughput and queue depth.

LLM quality metrics

- groundedness/citation adherence,
- instruction-following success,
- format compliance (e.g., valid JSON schema).

Safety and governance metrics

- policy violation rate,
- blocked response count,
- sensitive-content escalation rate.

Standardization and trace semantics

Telemetry standards like OpenTelemetry semantic conventions (including GenAI-oriented conventions) improve cross-tool consistency for traces and metrics.

Evaluation in Production

Pre-deployment evaluation

- golden test suites,
- adversarial prompts,
- policy and red-team scenarios,
- latency and cost load tests.

Post-deployment evaluation loop

1. Sample production interactions.
2. Score via automated evaluators + human audits.
3. Compare against baseline version.
4. Trigger rollback or improvement cycle when thresholds are breached.

Evaluation rubric example

- answer correctness/grounding,
- instruction adherence,
- harmful output avoidance,
- response usefulness,
- cost and latency budget adherence.

Governance and Risk Management

Governance pillars

1. **Transparency**: versioned prompts/models and audit logs.
2. **Accountability**: owner per service and escalation path.

3. **Safety**: policy filtering and abuse monitoring.
4. **Compliance**: data handling controls and review evidence.

Governance artifacts

- model/system cards,
- risk register,
- prompt/policy change logs,
- incident reports and postmortems,
- periodic governance review checklist.

Applying risk frameworks

Frameworks such as NIST AI RMF and the Generative AI profile can be operationalized as control checklists mapped to pipeline checkpoints (design, deployment, monitoring, incident response).

Cost and Capacity Management

Core cost drivers

- token volume,
- model size/provider tier,
- retrieval workload,
- tool-call frequency.

Cost guardrails

- per-request token caps,
- model routing tiers,
- cache hit-rate targets,
- budget-based alerting.

Capacity planning

Plan for:

- peak request bursts,
- model provider throttling limits,
- failover to backup models/providers,
- queueing and backpressure behavior.

Production Case Studies & Exceptions

Case 1: Enterprise RAG assistant outage

Failure:

- index refresh pipeline stalled silently.
- assistant used stale context for critical policies.

Fix:

- index freshness heartbeat and fail-closed behavior when freshness SLA breached.

Case 2: Customer-facing copilot quality regression

Failure:

- prompt update improved style but reduced factual adherence.

Fix:

- prompt release gating with groundedness regression tests before promotion.

Case 3 (Exception): Strict compliance workflow

Requirement:

- zero tolerance for unsupported free-form answers.

Decision:

- constrain outputs to retrieval-grounded templates and deterministic policy checks.

Interview Questions & Answers

1. **What should be monitored in an LLM app beyond latency and errors?**
   Prompt versions, retrieval quality, token usage, safety events, and output quality metrics.
2. **Why is tracing important for LLM systems?**
   It reveals multi-step behavior across prompts, tools, and retrieval dependencies.
3. **How do you evaluate LLM quality in production?**
   With automated evaluations, human sampling, and regression comparisons against baselines.
4. **What is a good rollback trigger for LLM apps?**
   Sustained degradation on quality/safety metrics or major SLA violations.
5. **How do you manage token cost explosions?**
   Token caps, prompt compaction, caching, and model routing by complexity.
6. **What is groundedness and why does it matter?**
   Degree to which output is supported by trusted context; it reduces hallucination risk.
7. **How do you detect retrieval failures?**
   Monitor relevance scores, citation mismatch rates, and freshness metrics.
8. **What is policy guardrailing?**
   Rule-based checks that block or transform unsafe/non-compliant outputs.
9. **How does LLM observability differ from classical API monitoring?**
   It requires semantic quality and safety signals in addition to infrastructure telemetry.
10. **What governance artifacts should every LLM service maintain?**
    Risk register, system card, change logs, incident history, and review checklist.
11. **How do you design LLM SLOs?**
    Combine latency/availability with quality and safety thresholds.
12. **What role does OpenTelemetry play in LLMOps?**
    Standardizes telemetry vocabulary for better interoperability and analysis.
13. **How do you handle provider outages for hosted LLMs?**
    Use retries, circuit breakers, fallback providers/models, and degraded-mode responses.
14. **When is stricter output constraint preferable to open generation?**
    In regulated or high-consequence workflows requiring deterministic compliance.

15. **What is an effective governance operating rhythm?**
    Continuous telemetry review + periodic risk audits + post-incident control updates.

References

- Coursera: Orchestrate, Analyze, and Evaluate AI Deployments: https://www.coursera.org/learn/orchestrate-analyze-and-evaluate-ai-deployments
- Coursera Duke LLMOps specialization: https://www.coursera.org/specializations/large-language-model-operations
- OpenTelemetry semantic conventions (GenAI-related updates): https://opentelemetry.io/docs/specs/semconv/
- OpenTelemetry GenAI attribute registry pointer: https://opentelemetry.io/docs/specs/semconv/registry/attributes/ge ai/
- NIST AI RMF overview: https://www.nist.gov/itl/ai-risk-management-framework
- NIST Generative AI Profile (AI 600-1 PDF): https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf
- IBM LLMOps overview for lifecycle framing: https://www.ibm.com/think/topics/llmops